DTU Health
Department of Health Technology

Enhanced Classification of SSVEP Signals Using Common Average Referencing and Support Vector Machines

Federica Maria Olivotto (s241759) - Paolo Fedele Di Giglio (s241767)

22053 - Principles of brain computer interface

Enhanced Classification of SSVEP Signals Using Common Average Referencing and Support Vector Machines

22053 - Principles of brain computer interface
January, 2025

By
Federica Maria Olivotto - s241759
Paolo Fedele Di Giglio - s241767

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.
Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Health Technology, Ørsteds Plads, Building 345C, 2800 Kgs. Lyngby Denmark
www.healthtech.dtu.dk/
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Abstract

This study focuses on improving Steady-State Visual Evoked Potential (SSVEP)-based Brain-Computer Interface (BCI) performance by enhancing signal processing and feature extraction techniques. The Common Average Reference (CAR) was applied to reduce common noise across EEG channels, improving the visibility of harmonic peaks associated with SSVEP signals. A bandpass filter was used to isolate the relevant frequency range, ensuring minimal distortion. Canonical Correlation Analysis (CCA) was employed to extract features by correlating EEG signals with reference sinusoidal waveforms at target frequencies and their harmonics. To further enhance classification accuracy, a Support Vector Machine (SVM) classifier was applied for multiclass classification. Experimental results demonstrated that CAR filtering significantly improved signal quality, increasing the Signal-to-Noise Ratio (SNR) and enhancing the classification accuracy. The study also investigated the effect of varying the EEG segment duration, revealing a trade-off between classification accuracy and processing speed, with optimal performance achieved at a 2.5-second time window. The combination of CAR, CCA, and SVM yielded the highest classification accuracy and Information Transfer Rate (ITR), confirming the advantages of this method for real-time BCI applications. Further research into subject-specific adaptations could enhance the robustness and accuracy of the approach, particularly in noisy environments.

Contents

Abstract ii

1 Introduction, Motivation and Related Works **2**

2 Methods **3**
2.1 Signal Acquisition 3
2.2 Data Preprocessing 4
2.3 Features extraction using CCA 6
2.4 Classification with Support Vector Machines (SVM) 8

3 Results **10**
3.1 Results for EEG Duration = 6 Seconds 10
3.2 Classification Analysis 12
3.3 Results by varying the time window 12

4 Discussions and Conclusions **14**
4.1 Future improvements: 14

Bibliography **17**

A Appendix: Code **19**

1 Introduction, Motivation and Related Works

Brain-Computer Interfaces (BCIs) represent an innovative technology that enables direct communication between the brain and external devices, bypassing traditional neuromuscular pathways, and providing patients with disabilities a means to communicate and interact with their environment. Among various BCI paradigms, steady-state visual evoked potentials (SSVEPs) have gained significant attention due to their high signal-to-noise ratio (SNR), high information transfer rate (ITR), and minimal user training requirements. SSVEPs are periodic neural responses elicited by visual stimuli steadily flickering at specific frequencies, typically observed in the occipital region of the brain. The intrinsic frequency specificity of SSVEPs makes them highly suitable for applications requiring multiple command options, such as spelling devices, in which each character flickers at a different frequency.

Our work builds on the 2015 study by Chen et al., titled “Filter bank canonical correlation analysis for implementing a high-speed SSVEP-based brain–computer interface” [1]. This study utilized Canonical Correlation Analysis (CCA) for SSVEP detection, a widely used method in SSVEP-based BCIs due to its efficiency, robustness, and simplicity. CCA identifies correlations between EEG signals and predefined reference templates representing visual stimulus frequencies. Chen et al. introduced the Filter Bank Canonical Correlation Analysis (FBCCA) method, which incorporates both fundamental and harmonic frequencies to enhance CCA’s performance in detecting SSVEPs. Their results demonstrated that FBCCA significantly outperforms standard CCA in terms of accuracy and Information Transfer Rate (ITR). However, FBCCA’s high computational cost, especially with an increased number of filter banks, poses challenges for real-time applications [2].

For this reason, our aim was to find an alternative method to enhance the performances of CCA in the identification of the target SSVEP. Specifically, we combined CCA, which was employed during the feature extraction phase, with a Support Vector Machine (SVM) classification algorithm. SVM is a widely used machine learning algorithm known for its robustness, generalization ability, and effectiveness in handling high-dimensional data [3]. Previous studies ([3], [4]) have demonstrated that integrating SVM with CCA can significantly improve the performance of traditional CCA-based methods. In standard CCA, the extracted features are correlation coefficients between EEG signals and predefined reference signals, and the highest correlation coefficient is selected to identify the target frequency. However, this ”maximum classifier” approach can lead to misclassification, particularly when working with short data lengths, making it difficult to achieve the desired level of recognition accuracy for target frequencies. Replacing the ”maximum classifier” with an SVM helps to improve the accuracy of target recognition [4].

Moreover, a Common Average Reference (CAR) filter was applied to enhance the performance of the classification algorithm. CAR is a spatial filtering technique widely used in electroencephalography (EEG). This method involves re-referencing the EEG signals, by subtracting the mean signal across all electrodes from the signal at each individual electrode [5]. We decided to apply a CAR filter even though the EEG signals were already referenced to the electrode Cz, in order to mitigate possible residual noise or common interference, such as muscle artifacts or electrical noise, that could still affect the recorded signals [5].

2 Methods

2.1 Signal Acquisition

The dataset used in this study is obtained from a 40-target BCI speller system, featuring 64-channel EEG data from 35 subjects. The data acquisition utilizes joint frequency and phase modulation (JFPM) for stimulus coding, allowing for comprehensive system evaluation and computational modeling in SSVEP-based BCIs. The data were originally acquired at 1000 Hz and then downsampled to 250 Hz. No additional digital filters were applied to the dataset, apart from a 50 Hz notch filter to remove power-line noise. For further details, refer to the benchmark paper [6]. An example of the data signal acquisition scheme is reported in Figure 2.1.

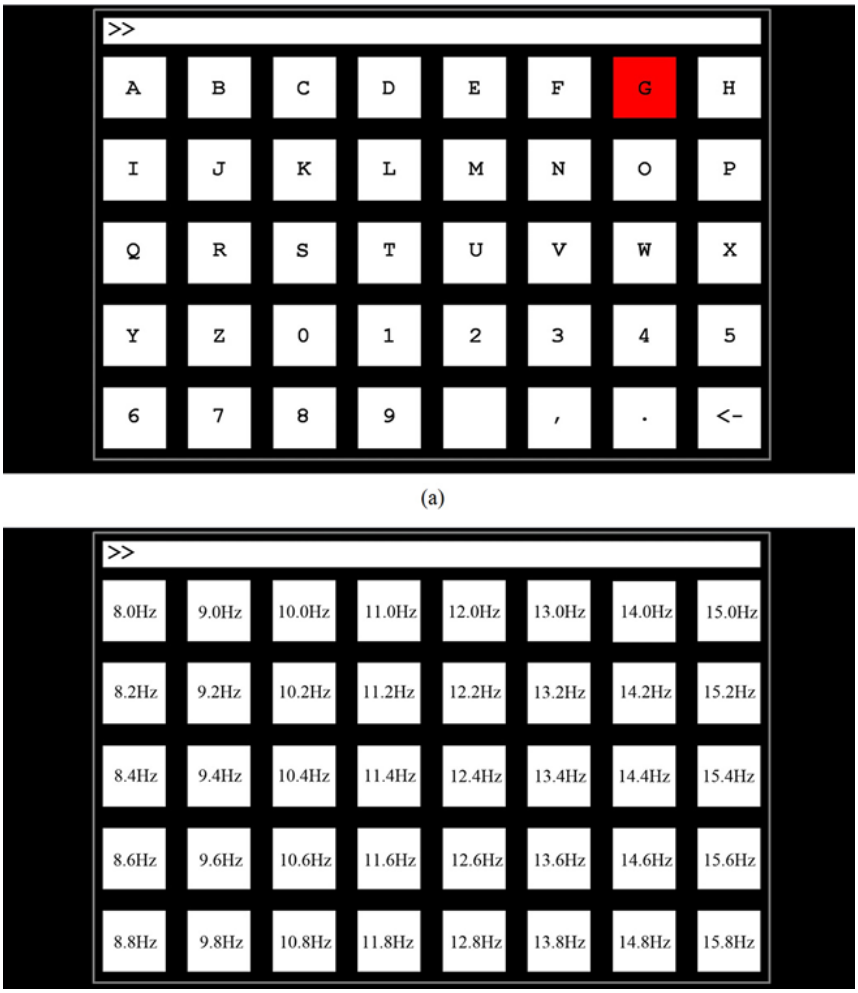


Figure 2.1: (a) The user interface of the 40-target BCI speller. The red square is an example of a visual cue indicating a target character 'G' in the BCI experiments. (b) The frequency values for all characters

2.1.1 Dataset Structure

The dataset consists of data from 35 subjects, with each subject providing a 4D data matrix of dimensions [64, 1500, 40, 6], where the indices represent the following: "Electrode

index," "Time points," "Target index," and "Block index." In terms of single trials, the data matrix includes 240 trials (40 targets $\times$ 6 blocks), with each trial containing 64 channels of 1500-point data. The original trial duration is 6 seconds, sampled at 250 Hz, resulting in 1500 time points (samples). This time frame includes 0.5 seconds before stimulus onset, 5 seconds of stimulation, and 0.5 seconds after stimulus offset.

2.2 Data Preprocessing

2.2.1 Electrode Selection

We selected the 9 electrodes Pz, PO5, PO3, POz, PO4, PO6, O1, Oz, and O2, as indicated in the reference paper [1], for their proven effectiveness in SSVEP analysis. These electrodes cover the occipital and parieto-occipital areas, which are associated with the primary and secondary visual cortices, generating robust SSVEP signals with a high signal-to-noise ratio. The chosen configuration reduces noise, captures laterally distributed components, improves robustness against artifacts, and optimizes the effectiveness of algorithms like CCA. Widely validated in the literature, it is a standard compatible with different stimulating frequencies, ensuring high generalization across various experimental paradigms. [7]

2.2.2 Selection of the Important signal

As shown in Figure 2.2, we trimmed each EEG signal by removing the first and last 0.5 seconds, along with an additional 0.14-second delay, to analyze only the significant portions of the signal. This delay reflects the physiology of the SSVEP response, helping avoid noise or artifacts caused by the brain's latency in processing the stimulus.

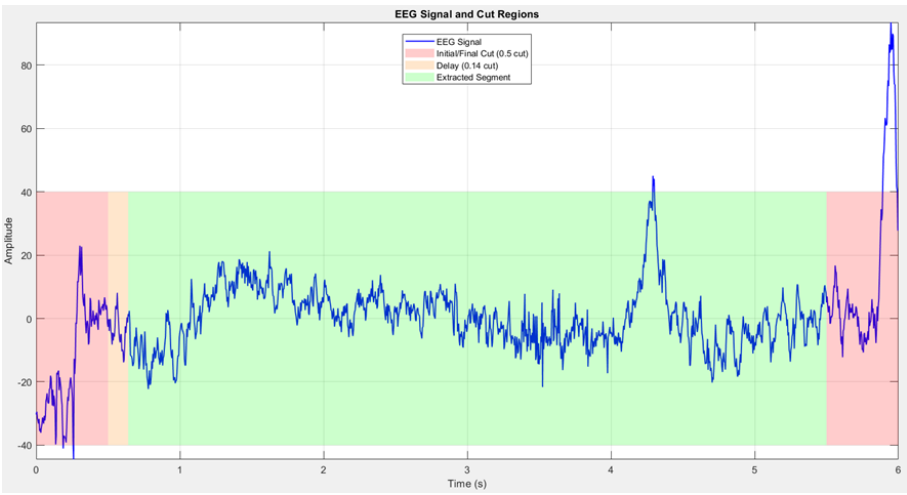
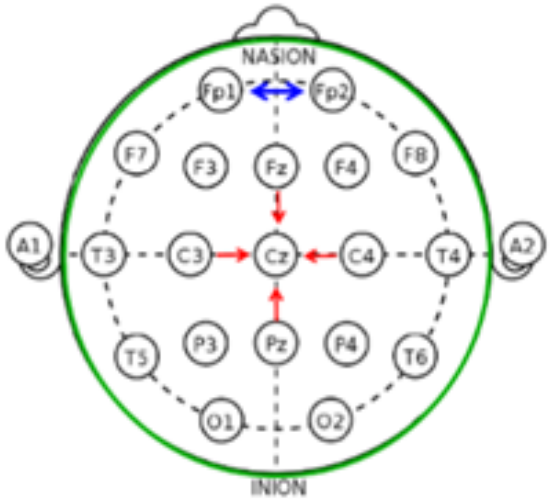


Figure 2.2: Example of Cutted EEG Signal

2.2.3 Referencing

We applied the Common Average Reference (CAR) filter to reduce common noise and improve the discrimination of SSVEP harmonic peaks. As shown in Fig 2.3, the CAR computes the signal average across all electrodes and subtracts it from each electrode's signal, thereby improving overall quality.



Re-referencing: $$\hat{\mathbf{e}}_i(t) = \mathbf{e}_i(t) - \frac{1}{N}\sum_{i=1}^{N}\mathbf{e}_i(t)$$

Figure 2.3: CAR Formula

2.2.4 Band-Pass Filter

After referencing, a bandpass filter (5-90 Hz), shown in Figure 2.4, was applied to the EEG signal to isolate the frequencies of interest, improving signal quality for harmonic SSVEP analysis. The 90 Hz upper limit includes up to the fifth harmonic (79 Hz), while the 5 Hz lower limit removes low-frequency physiological artifacts, such as the eye-blink artifact, which is typically in the range 0.1 Hz - 4 Hz. Initially, the bandpass filter was applied before the CAR filter, but this caused distortions in referencing. The order of application was then corrected, and a 15th-order Butterworth filter was used to ensure a flat frequency response in the interest band, minimizing distortions in the critical frequencies.

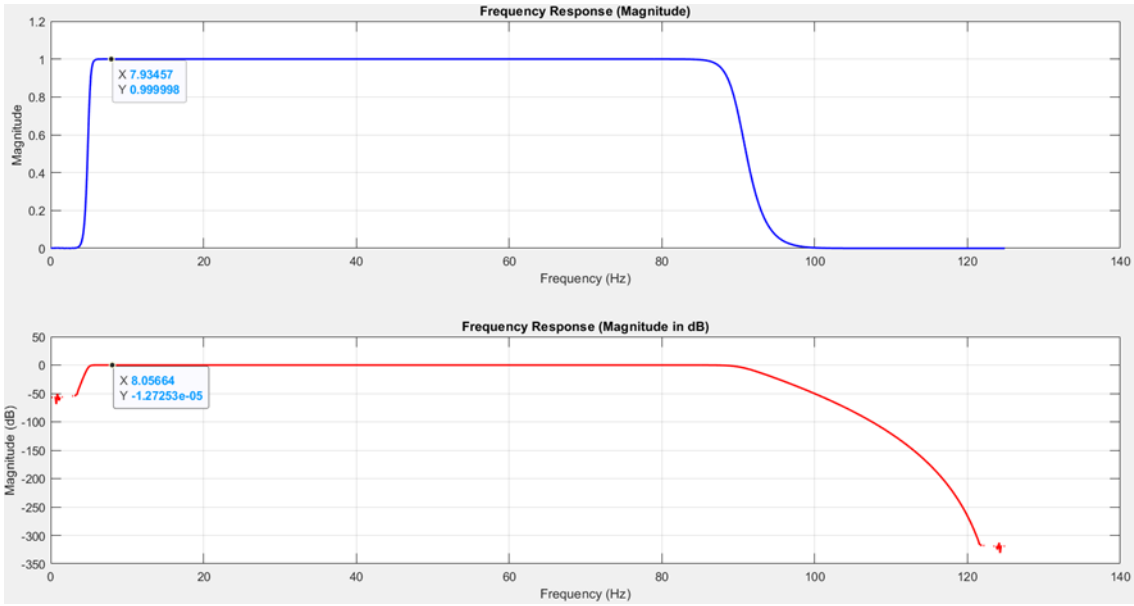


Figure 2.4: Frequency response of the band pass filter

$$Y_f = \begin{bmatrix} \sin(2\pi ft) \\ \cos(2\pi ft) \\ \vdots \\ \sin(2\pi N_h ft) \\ \cos(2\pi N_h ft) \end{bmatrix}$$

Figure 2.5: Sinusoidal reference signals for the stimulation frequency f

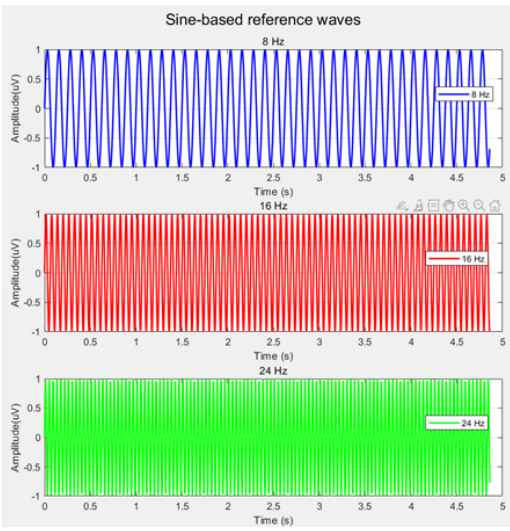
2.3 Features extraction using CCA

Canonical Correlation Analysis (CCA) is employed to identify the frequency of visual stimuli that the user is focusing on, by analyzing the correlation between recorded multichannel EEG signals and reference signals constructed at the stimulus frequencies.

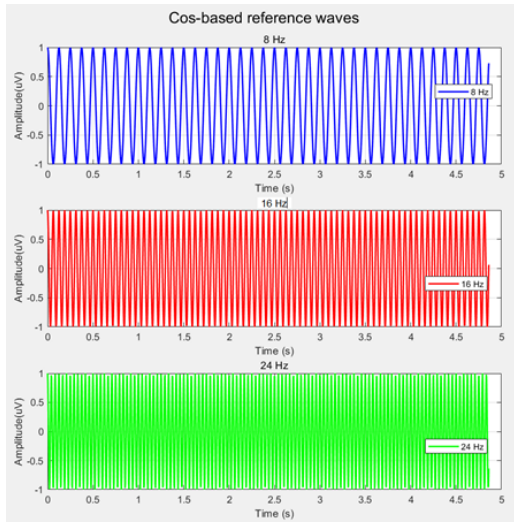
2.3.1 Reference Signals Generation

Reference signals Y_f, shown in Figure 2.5, are constructed for each of the 40 stimulation frequencies (f) and their harmonic components, using sinusoidal waveforms, specifically sine and cosine waves. In this case, five harmonics were considered for each stimulation frequency ($N_h = 5$), so the reference signal vector was composed of a total of $40 \times 5 \times 2 = 400$ sinusoidal waves. The length of the reference signals was the same as the EEG signals (4.86 s), in order to compare them sample by sample.

The plot of the resulting reference signals for the stimulation frequency $f = 8$ Hz and the first two harmonic components (16 Hz and 24 Hz) is shown in Figure 2.6.



(a) Sine Reference waves.



(b) Cosine Reference waves.

Figure 2.6: Plot of the sinusoidal reference signals, sine (left) and cosine (right), obtained for the stimulation frequency $f = 8$ Hz and for its first two harmonic components (16 Hz and 24 Hz).

2.3.2 Canonical Correlation Analysis

In our implementation, we utilized a custom function, `myCCA`, to perform Canonical Correlation Analysis (CCA). This function processes two inputs: the multichannel EEG data matrix corresponding to a single data segment and the matrix of reference signals. Specifically, for each iteration, the EEG data segment comprising signals from 9 channels (denoted as X) is compared against a set of 10 sinusoidal reference signals (Y), which are generated by a specific stimulation frequency and its first 5 harmonics.

The CCA method considers the two linear combinations of the input signals:

$$x = X^T W_X \quad \text{and} \quad y = Y^T W_Y$$

and finds the weight vectors W_X and W_Y which maximize the correlation between x and y, by solving the following optimization problem:

$$\max_{\mathbf{W}_X, \mathbf{W}_Y} \rho = \frac{\mathbf{W}_X^T \mathbf{X}\mathbf{Y}^T \mathbf{W}_Y}{\sqrt{\mathbf{W}_X^T \mathbf{X}\mathbf{X}^T \mathbf{W}_X \cdot \mathbf{W}_Y^T \mathbf{Y}\mathbf{Y}^T \mathbf{W}_Y}}$$

Where $\mathbf{XX}^T$, $\mathbf{YY}^T$, $\mathbf{XY}^T$ are respectively the covariance matrices of X and Y, and the cross-covariance matrix between X and Y.

The function `myCCA` solves the canonical correlation problem via a generalized eigenvalue problem:

$$(\mathbf{XX}^{-1} \cdot \mathbf{XY} \cdot \mathbf{YY}^{-1} \cdot \mathbf{YX}) W_X = \lambda W_X$$

Where W_X and W_Y are the canonical weight vectors, corresponding to the directions of maximum correlation for X and Y, and λ are the eigenvalues, related to the canonical correlations as $r = \sqrt{\lambda}$. Finally, the canonical correlations and their corresponding weight vectors are sorted in descending order of ρ and are returned as output by the function.

This process is repeated for each data segment to compare it with all 40 stimulation frequencies and their corresponding harmonics. Furthermore, the CCA procedure is applied to all trials (240 trials per subject) across all 35 subjects. At the end of this process, a vector called `Rho` is saved for each subject and trial, containing the 40 correlation coefficients between the corresponding EEG segment and each of the possible stimulation frequencies. These `Rho` vectors are then used as features in the subsequent classification phase.

2.3.3 Frequency Identification

At the conclusion of the CCA process, a simple maximum-correlation approach could be used to predict the stimulation frequency without using a classification algorithm. In this method, the highest canonical correlation is identified, and the corresponding stimulation frequency is selected as the detected frequency, as it represents the frequency most strongly reflected in the EEG data.

However, this approach has several limitations: it is highly sensitive to noise and prone to errors when the EEG data segments are short. Additionally, it relies on the most similar frequency only, disregarding the broader set of harmonic components that may carry valuable information.

To address these challenges, we implemented a classification algorithm that utilizes the richer information contained in the full `Rho` vector, which captures the correlation profile across all stimulation frequencies, and we investigated its advantages compared to the simple maximum-correlation approach.

2.4 Classification with Support Vector Machines (SVM)

One of the most popular classification algorithms is the Support Vector Machine (SVM), which aims to find the optimal hyperplane that separates two classes by maximizing the margin between them. Assuming that the data is linearly separable, there exist multiple hyperplanes that solve the classification problem. SVMs choose the one that maximizes the margin, assuming that this will generalize better to new unseen data [8].

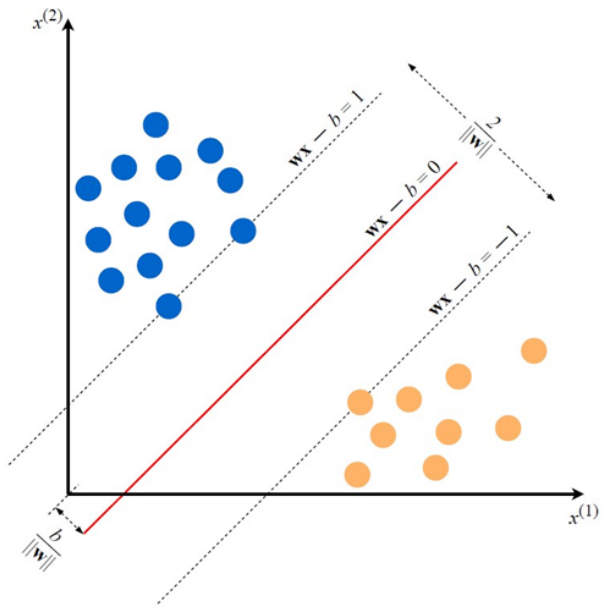


Figure 2.7: Hyperplane in SVM [**ref2**].

Our problem is a multi-class classification problem, as we need to distinguish between 40 possible target frequencies. We used the Error-Correcting Output Codes (ECOC) model, which combines multiple binary SVM classifiers to handle multi-class classification problems.

Steps of the Classification Phase

Splitting into Train and Test Sets

The dataset, obtained after the feature extraction phase and consisting of all the `Rho` vectors corresponding to each trial for each patient, was divided into a training set and a test set. Specifically:

- The first 200 trials (initial 5 blocks) were allocated to the training set.
- The remaining 40 trials (final block) were used for testing.

This approach ensured that data from all 35 patients were included in both the training and test sets, enabling evaluation on the same patient population used for training. This design is consistent with typical BCI applications, which prioritize performance for specific individuals or groups over broader generalization across populations.

Training Phase

The model was trained using MATLAB's `fitcecoc` function, employing a 5-fold cross-validation approach to assess its performance. During this phase, cross-validation accuracy was calculated to indicate the model's generalization capability. After cross-validation, the final model was trained on the entire training set without applying cross-validation.

Test Phase

The `predict` function in MATLAB was used to apply the trained model to the test dataset.

Performance Metrics

Qualitative Evaluation

We performed a qualitative evaluation of the improvements introduced by the CAR filter by:

- Plotting the power spectrum, calculated using the Fast Fourier Transform (FFT), for a single trial and for the trials averaged across patients and blocks.
- Calculating and plotting the corresponding Signal-to-Noise Ratios (SNR) in decibels (dB). The SNR was defined as the ratio of y(f) to the mean value of the 10 neighboring frequencies (i.e. five frequencies on each side).

$$\text{SNR} = 20 \log_{10} \frac{y(f)}{\sum_{k=1}^{5} [y(f - 0.2 \times k) + y(f + 0.2 \times k)]}$$

Classification Accuracy

The classification accuracy was computed by comparing the predicted frequency labels with the target frequency labels, counting the total number of correct predictions, and dividing by the total number of trials. Additionally, overfitting was assessed by comparing test accuracy with cross-validation accuracy. A lower test accuracy compared to cross-validation accuracy indicated potential overfitting.

Information Transfer Rate (ITR)

The Information Transfer Rate (ITR) was estimated using the following formula:

$$\text{ITR} = \frac{60}{T} \cdot \left[\log_2(M) + P \log_2(P) + (1 - P) \log_2 \left(\frac{1 - P}{M - 1} \right) \right]$$

where:

- M: Number of targets (40).
- P: Classification accuracy.
- T: Average time required for a selection (in seconds). Since the algorithm was not tested in a real online phase, T was assumed proportional to the duration of each data segment, ranging from 2 to 6 seconds.

Additional Metrics

Additional performance metrics included:

- **Confusion Matrix**: Visualizes correct predictions and misclassifications for each target class.
- **ROC Curve**: Evaluates the classifier's performance for each class by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) across different thresholds.
- **Area Under the Curve (AUC)**: Computed for each target class to summarize classifier performance.

3 Results

Initially, we used the full length of the EEG data segments (6 seconds initially, reduced to 4.86 seconds after removing the delay and trimming the first and last 0.5 seconds) to develop our classification algorithm and evaluate its performance. Subsequently, we repeated the procedure with shorter EEG segment lengths to simulate a more realistic real-time application, where minimizing the time required to recognize each target is critical.

3.1 Results for EEG Duration = 6 Seconds

3.1.1 Single Trial Results

The following plots were generated for Patient 1, electrode Pz, during the first trial of Block 1, which corresponds to a stimulation frequency of 8 Hz.

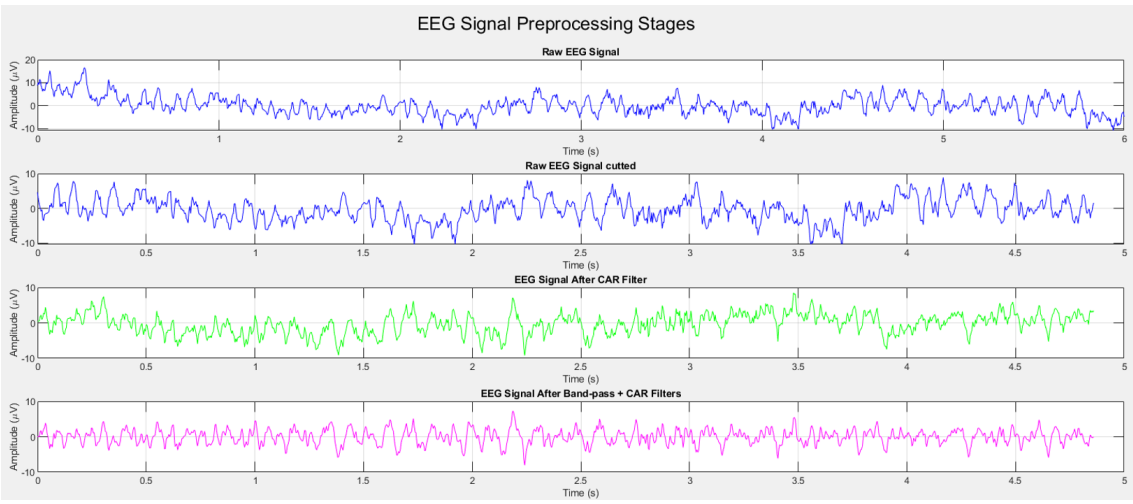


Figure 3.1: Raw EEG segment (1), EEG after trimming and delay removal (2), signal after applying the CAR filter (3), and signal after applying the band-pass filter (4). The CAR filter reduces low-frequency oscillations and centers the signal around the baseline, while the band-pass filter removes low-frequency drifts.

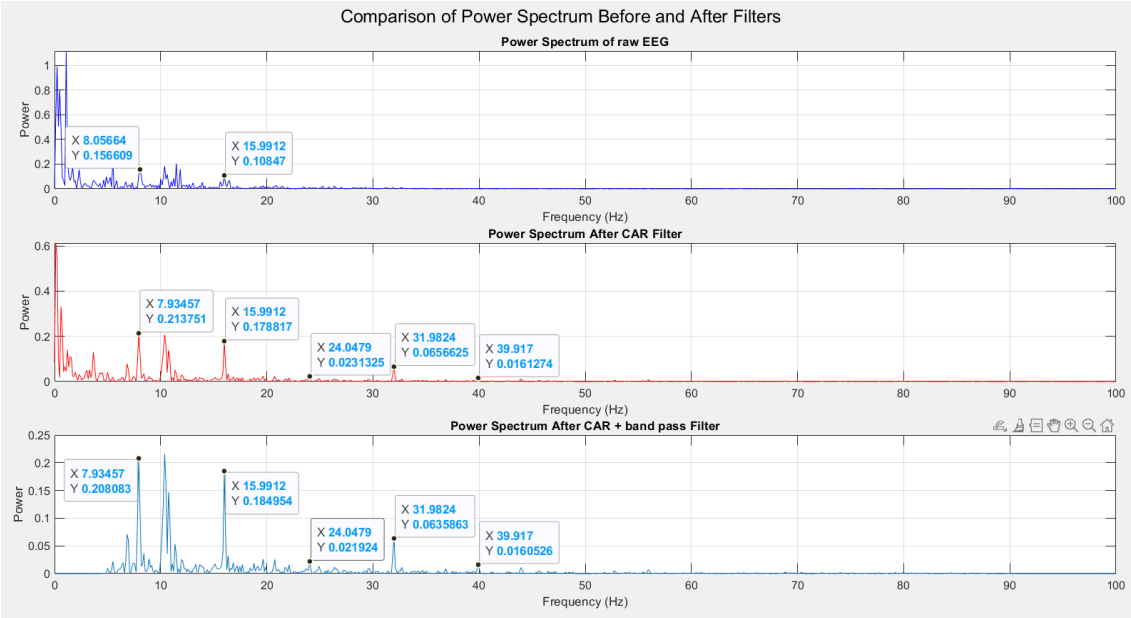


Figure 3.2: Power spectra of the trimmed EEG signal (1), signal after applying the CAR filter (2), and signal after applying the band-pass filter (3). The CAR filter enhances peaks at 8 Hz and its harmonics (16 Hz, 24 Hz, 32 Hz, 40 Hz). Notable peaks outside these frequencies (around 10 Hz) are discussed in the discussion section

3.1.2 Average Results

The following plots depict the signal obtained by averaging the EEG data across all blocks and patients while keeping the electrodes and stimulation frequencies separate. Specifically, these plots show the average signal at electrode Pz for a stimulation frequency of 8 Hz.

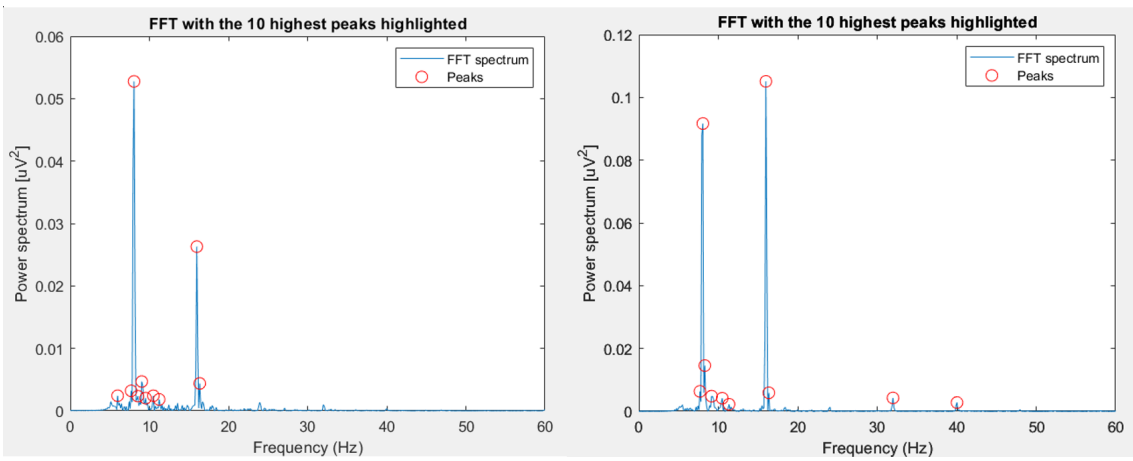


Figure 3.3: Power spectrum of the averaged EEG signal without (left) and with (right) the CAR filter. The 10 highest peaks in the power spectra are highlighted with red circles. The application of the CAR filter further enhances the detection of the frequency components of interest. For example, the amplitude of the 8 Hz peak increases from $0.053\,\mu\text{V}^2$ to $0.092\,\mu\text{V}^2$. Furthermore, with the CAR filter applied, the harmonics at 32 Hz and 40 Hz are among the 10 highest peaks, whereas they are not identified as such without the filter.

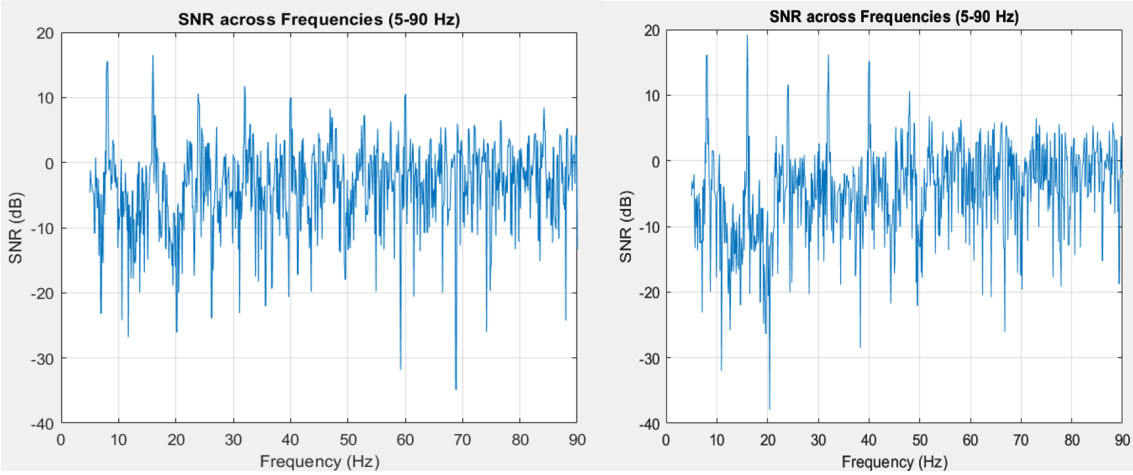


Figure 3.4: Signal-to-Noise Ratio (SNR) of the averaged EEG signal without (left) and with (right) the CAR filter. The SNR amplitude at 8 Hz increases from 15.5 dB to 16.1 dB after applying the filter.

3.2 Classification Analysis

Upon applying Common Average Referencing (CAR) to the dataset, the peaks corresponding to fundamental and harmonic frequencies became more distinct. This led us to compare the performance of the developed methods, assessing the utility of CAR and SVM. As shown in the table, the results clearly indicate an advantage in all metrics when using CAR referencing. This is likely due to CAR's ability to remove common noise across the electrodes, thereby enhancing classification performance. Similar improvements are observed with the use of SVM. In conclusion, the CAR-stCCA-SVM method is preferred.

Table 3.1: Accuracy and ITR results with EEG duration of 2.5s.

Method	Accuracy	Mean AUC (over classes)	ITR
noCAR-stCCA	$76.87 \pm 3.68\%$	—	56.90
CAR-stCCA	$79.26 \pm 3.39\%$	—	59.82
noCAR-stCCA-SVM			
Train Accuracy	80.53%	—	—
Test Accuracy	82.14%	0.9823 ± 0.0081	63.45
CAR-stCCA-SVM			
Train Accuracy	81.91%	—	—
Test Accuracy	83.79%	0.9836 ± 0.0071	65.58

3.3 Results by varying the time window

The time window refers to the duration of the EEG signal, excluding the first and the last 0.5 seconds. A comparative analysis of the CAR-stCCA and CAR-stCCA-SVM models was conducted to evaluate performance across different time windows, considering accuracy and ITR. As shown in the figure 3.5, a tradeoff between accuracy and model speed is evident, with both models performing optimally with a time window of 2.5 seconds. Furthermore, the results demonstrate an improvement in both metrics for the CAR-stCCA-SVM model. The integration of SVM improves both accuracy and robustness, leading to an increase in ITR.

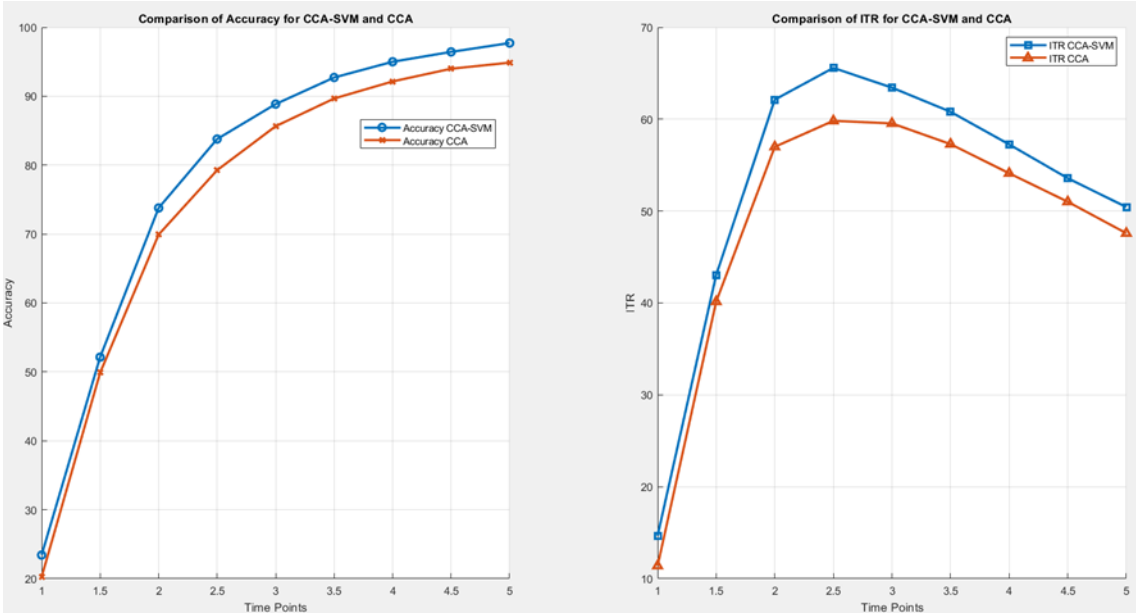


Figure 3.5: Comparison of CAR-stCCA and CAR-stCCA-SVM performance with varying time windows

4 Discussions and Conclusions

This study highlights key advancements in SSVEP-based BCI performance. The Support Vector Machine (SVM) algorithm outperformed standard Canonical Correlation Analysis (CCA) in classification accuracy, demonstrating the benefits of machine learning approaches. Additionally, the application of the Common Average Reference (CAR) filter improved Signal-to-Noise Ratio (SNR) and enhanced the detection of stimulation frequencies and their harmonics. These findings underscore the combined importance of advanced classification methods and effective preprocessing for robust BCI performance.

Despite these promising results, several limitations must be acknowledged. The algorithm was tested exclusively on the Benchmark dataset, where EEG data were collected under controlled conditions. To comprehensively evaluate its performance, it is crucial to test the system in real-time applications, where external factors such as noise and user behavior may significantly vary. An improvement could involve training and testing the algorithm on datasets augmented with noise, thereby enhancing the model's robustness and performance in real-time scenarios. A relevant example is provided in the study by [2]

Furthermore, while calculating the Information Transfer Rate (ITR), we assumed that the time per selection (T) was proportional to the duration of the data segment. This assumption is reasonable for offline analysis but may not hold in real-time BCI systems, where factors like system latency, user response time, and processing time contribute to T. Testing the algorithm in a real-time context would provide a more accurate assessment of its ITR and feasibility for practical applications.

4.1 Future improvements:

- In the power spectrum plots, we observed unexpected peaks at frequencies near the stimulation frequency (e.g., around 9 Hz and 10 Hz for a stimulation frequency of 8 Hz). These peaks did not correspond to the stimulation frequency or its harmonics. We hypothesize that these peaks are caused by other characters on the screen, positioned near the target the patient is focusing on, which remain within the patient's visual field and generate weaker SSVEP responses. A potential improvement to mitigate these disturbances could be increasing the spacing between characters on the screen.
- An alternative strategy to the reference electrode Cz, proposed by Wu and Su (2014), involves dynamically selecting the reference electrode to maximize the signal-to-noise ratio (SNR), optimizing the referencing effectiveness in SSVEP-based BCI systems. [9]
- Individual templates (IT): Introduce a patient-specific calibration phase, in which the EEG signals generated by the user in response to each stimulation frequency are recorded and used as the reference signals in the CCA comparison, instead of the artificially generated templates. This solution was implemented in the study [10] and showed significant improvements in the system performances. Another approach could involve replacing the current machine learning model with a more advanced one capable of updating its parameters dynamically during use. For example, as highlighted in [11], the Google Titans model leverages a "long and adaptable memory module" to learn contextual patterns and adjust to changes in these patterns

over time. This functionality could greatly enhance the method by enabling greater personalization and making it dynamically patient-specific. However, this remains a preliminary proposal, as the relevant research only emerged in December 2024. Implementing such a model would require a tailored adaptation for SSVEP tasks, which are fundamentally different from the typical applications of Large Language Models.

- Leveraging the fact that SVM-based classifiers produce a score that indicates its confidence for the label of the testing trial, it could be possible to set a minimum threshold of confidence that can be used as an early stopping criterion (i.e. the user looks at the stimuli for the time required by the classifier to reach the minimum threshold). Eventually, each user would need to focus on the stimuli only for the required time to reach the threshold, resulting in user-dependent varying trial lengths and in turn in higher Information Transfer Rate (ITR). Conversely, the time duration of each trial was fixed and equal to 1.8 s in the online experiment of the paper [1]. This improvement has already been implemented in the study [3].

Bibliography

[1] Xiaogang Chen et al. “Filter bank canonical correlation analysis for implementing a high-speed SSVEP-based brain–computer interface”. In: *Journal of neural engineering* 12.4 (2015), p. 046008.

[2] Changkai Tong, Huali Wang, and Jun Cai. “A novel turbo detector design for a high-speed SSVEP-based brain speller”. In: *Electronics* 11.24 (2022), p. 4231.

[3] Elisavet Chatzilari et al. “Combining the benefits of CCA and SVMs for SSVEP-based BCIs in real-world conditions”. In: *Proceedings of the 2nd International Workshop on Multimedia for Personal Health and Health Care*. 2017, pp. 3–10.

[4] Pengfei Ma et al. “A classification algorithm of an SSVEP brain-Computer interface based on CCA fusion wavelet coefficients”. In: *Journal of Neuroscience Methods* 371 (2022), p. 109502.

[5] Xiaowei Zheng et al. “Enhancing performance of SSVEP-based visual acuity via spatial filtering”. In: *Frontiers in Neuroscience* 15 (2021), p. 716051.

[6] Yijun Wang et al. “A benchmark dataset for SSVEP-based brain–computer interfaces”. In: *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 25.10 (2016), pp. 1746–1752.

[7] YB Eisma et al. “Assessing the Influence of Visual Stimulus Properties on Steady-State Visually Evoked Potentials and Pupil Diameter”. In: (2024).

[8] URL: https://rubikscode.net/2021/02/15/machine-learning-with-ml-net-support-vector-machines/.

[9] Zhenghua Wu and Sheng Su. “A dynamic selection method for reference electrode in SSVEP-based BCI”. In: *PLoS One* 9.8 (2014), e104248.

[10] Guangyu Bin et al. “A high-speed BCI based on code modulation VEP”. In: *Journal of neural engineering* 8.2 (2011), p. 025015.

[11] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. “Titans: Learning to Memorize at Test Time”. In: *arXiv preprint arXiv:2501.00663* (2024).

A Appendix: Code

```
%% Initialization
clc;
clear;
close all;

%% Parameters
Fs = 250;                    % Sampling rate (Hz) BENCHMARKDATASET
Nh = 5;                      % Number of harmonics to considerato for the reference signal
duration = 3.5;                % Duration of signal we want to analyze (include pre & after stimulus) [seconds]
duration_raw_dataset= 6;     %Entire length of the raw signal [seconds]
cut_time = 0.5;              % time used to cut the signal pre & after stimulus
delay = 0.14;                 % Delay time due to neural pathways and creation of the SSVEP
duration_ref = duration-cut_time*2-delay;   % Cutted EEG duration time to analyze [seconds]
Len = duration * Fs;                          % # samples of the entire EEG
t_raw = linspace(0, duration_raw_dataset, duration_raw_dataset * Fs); % time pint cutted EEG
t = linspace(0, duration, Len);                                       % time points entire EEG
Len_ref = round(duration_ref * Fs);                                   % # samples of the Cutted EEG
t_ref = linspace(0, duration_ref, Len_ref);                           % time points of the Cutted EEG
svm_enabled = input('Do you want to enable SVM? (1 for Yes, 0 for No): ');  % Used to enable/disable SVM and use only standard-CCA

channels = [48 54 55 56 57 58 61 62 63];                                      % Channels used in this study: Pz, PO5, PO3, POz, PO4, PO6, O1, Oz, O2 -> 9 channels
n_channels = length(channels);
frequency_band = [5 90];     % Bandpass filter frequency range (Hz)
n_subjects = 35;             % Number of subjects
n_runs = 240;                % Number of runs per subject (40 trials x 6 blocks)
n_classes = 40;              % Number of classes
n_blocks = 6;

%% (  Extraction and creation of the dataset  )
% Load of datas in directory
data_dir = "C:\Users\Paolo\OneDrive - IIS CARLO LEVI - SANT'ARCANGELO\Desktop\DTU programs\Principles of Brain Computer Interface\MiniProject\SSVEP-BCI-Data\SSVEP-BCI-Data";
% Get a list of all .mat files in the directory
file_list = dir(fullfile(data_dir, 'S*.mat'));

% Preallocate for storing data
all_subjects_data = cell(length(file_list), 1);

% Loop through files
for i = 1:length(file_list)
    % Load each .mat file
    file_path = fullfile(data_dir, file_list(i).name);
    loaded_data = load(file_path);

    % Store the 'data' matrix in a cell array
    all_subjects_data{i} = loaded_data.data;
end

% Access data for subject 1
subject1_data = all_subjects_data{1};
disp(size(subject1_data)); % Output: [64, 1500, 40, 6]

%% Selection of the 9 electrodes
% Preallocate for storing data
Nine_channels_data = cell(length(file_list), 1);

% Loop through all subjects and modify the data to keep only selected electrodes
for i = 1:length(all_subjects_data)
    % Get the data for the current subject
    subject_data = all_subjects_data{i};

    % Extract the data corresponding to the selected electrodes
    % The 'extracted_indexes' holds the indices of the target electrodes
    subject_data_selected = subject_data(channels, :, :, :);  % Select electrodes

    % Store the filtered data back into the cell array (or you could save it)
    Nine_channels_data{i} = subject_data_selected;
end

% Example: Access modified data for subject 1 (only selected electrodes)
Nine_channels_data1 = Nine_channels_data{1};
disp(size(Nine_channels_data1));  % This will output: [9, 1500, 40, 6] for the 9 selected electrodes

%% Cutted Dataset --> Remotion of pre-stimulus & post stimulus parts (0.5 seconds each) + Remotion of delay time (first 0.14 seconds of after the stimulus start)
new_data = cell(length(file_list), 1);

for sbj = 1:length(Nine_channels_data)
    % Get the data for the current subject
    subject_data = Nine_channels_data{sbj};

    % Concatenate trials across all blocks
    %EEGdata = concatenate_runs(subject_data); % [9 x 1500 x 240]

    % loop over all blocks
    for block = 1:n_blocks
        EEGdata = subject_data(:,:,:,block);
```

```
        for trial = 1:n_classes
            % Extract EEG data for the current run
            X = EEGdata(:, :, trial); % [9 x 1500]

            % Extract the segment of X for the relevant time window
            X_segment = X(:,find(t >= cut_time & t <= cut_time + duration_ref)); % [9 x time points]

            new_data{sbj}(:,:,trial,block)=X_segment;
        end
    end
end

% Example: Access modified data for subject 1 (only selected electrodes)
new_data1 = new_data{1};
disp(size(new_data1));  % This will output: [9, 1215, 40, 6]

%% Create reference signals for each class
% Load stimulus frequencies and phases
load('Freq_Phase.mat');
fstim = freqs;
clear freqs;

%Reference signal generation
Xref = generate_reference_signals(fstim, Nh, t_ref);

% Plot the first three harmonics
figure(1);
hold on;
subplot(3,1,1);
plot(t_ref, Xref(:,1,1), 'b', 'LineWidth', 1.5);  % 1st Harmonic, 1st Class
subtitle('8 Hz');
xlabel('Time (s)');
ylabel('Amplitude(uV)');
grid on;
legend('8 Hz', 'Location', 'best');  % Add legend here

subplot(3,1,2);
plot(t_ref, Xref(:,3,1), 'r', 'LineWidth', 1.5);  % 2nd Harmonic, 1st Class
subtitle('16 Hz');
xlabel('Time (s)');
ylabel('Amplitude(uV)');
grid on;
legend('16 Hz', 'Location', 'best');  % Add legend here

subplot(3,1,3);
plot(t_ref, Xref(:,5,1), 'g', 'LineWidth', 1.5);  % 3rd Harmonic, 1st Class
subtitle('24 Hz');
xlabel('Time (s)');
ylabel('Amplitude(uV)');
grid on;
legend('24 Hz', 'Location', 'best');  % Add legend here
hold off;

sgtitle("Sine-based reference waves");

figure(2);
hold on;
subplot(3,1,1);
plot(t_ref, Xref(:,2,1), 'b', 'LineWidth', 1.5);  % 1st Harmonic, 1st Class
subtitle('8 Hz');
xlabel('Time (s)');
ylabel('Amplitude(uV)');
grid on;
legend('8 Hz', 'Location', 'best');  % Add legend here

subplot(3,1,2);
plot(t_ref, Xref(:,4,1), 'r', 'LineWidth', 1.5);  % 2nd Harmonic, 1st Class
subtitle('16 Hz');
xlabel('Time (s)');
ylabel('Amplitude(uV)');
grid on;
legend('16 Hz', 'Location', 'best');  % Add legend here

subplot(3,1,3);
plot(t_ref, Xref(:,6,1), 'g', 'LineWidth', 1.5);  % 3rd Harmonic, 1st Class
subtitle('24 Hz');
xlabel('Time (s)');
ylabel('Amplitude(uV)');
grid on;
legend('24 Hz', 'Location', 'best');  % Add legend here
hold off;

sgtitle("Cos-based reference waves");

%% Common Average Referencing (CAR) Application
CAR_data = cell(length(file_list), 1);

for sbj = 1:n_subjects
    subject_data = new_data{sbj};

    for block = 1:n_blocks
        EEGdata = subject_data(:,:,:,block);

        for trial = 1:n_classes
            X = EEGdata(:,:,trial)';

```

```
            % CAR filter
            CAR_data{sbj}(:,:,trial,block) = apply_car_filter(X)';
        end
    end
end

CAR_data1 = CAR_data{1};
disp(size(CAR_data1));  % This will output: [9, 1215, 40, 6] for the 9 selected electrodes

%% BAND-Pass Filtering

% ---> Design of the filter
[b, a] = butter(15, frequency_band / (Fs / 2), 'bandpass');  % designs an IIR (Infinite Impulse Response) Butterworth filter of the third order
                                                              % frequency_band / (Fs / 2): normalizes the passband [8, 88] Hz to [8/(Fs/2), 88/(Fs/2)]
                                                              % b and a: The filter coefficients for the numerator and denominator of the filter's transfer function
% Frequency Response using freqz
[H, f] = freqz(b, a, 1024, Fs); % Compute frequency response

% Plot Magnitude Response
figure(3);
subplot(2, 1, 1);
plot(f, abs(H), 'b', 'LineWidth', 1.5); % Magnitude in linear scale
title('Frequency Response (Magnitude)');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
grid on;

% Plot Magnitude in Decibels
subplot(2, 1, 2);
plot(f, 20*log10(abs(H)), 'r', 'LineWidth', 1.5); % Magnitude in dB
title('Frequency Response (Magnitude in dB)');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;

% ----> Application of BP-Filter on the dataset
band_pass_data = cell(length(file_list), 1);

for sbj = 1:n_subjects
    subject_data = CAR_data{sbj};

    for block = 1:n_blocks
        EEGdata = subject_data(:,:,:,block);

        for trial = 1:n_classes
            X = EEGdata(:,:,trial)';

            % Band-pass filter
            band_pass_data{sbj}(:,:,trial,block) = filtfilt(b, a, X)';
        end
    end
end

filtered_data = band_pass_data;
filtered_data1 = filtered_data{1};
disp(size(filtered_data1));  % This will output: [9, 1215, 40, 6] for the 9 selected electrodes

%% Extract signals for plots of the different stage of Pre-Processing(SEE LATER)
% Extract the signals for each stage (for example, for electrode 1)
electrode_signal = Nine_channels_data{1}(1, :, 1,1); % Original signal (before filtering)
electrode_signal1 = new_data{1}(1, :, 1,1); % Original signal (before filtering)
CAR_signal = CAR_data{1}(1,:,1,1); % CAR filtered signal
filtered_signal = filtered_data{1}(1, :, 1,1); % Band-pass + CAR filtered signal


%% FFT plots Creation
nfft = 2^nextpow2(length(electrode_signal)); % Zero-padding for FFT
frequencies = (0:nfft/2-1) * (Fs / nfft);    % Frequency axis (0 to Nyquist)

% FFT of the original signal
fft_original = fft(electrode_signal, nfft) / length(electrode_signal);
power_original = abs(fft_original(1:nfft/2)).^2; % Power spectrum

% FFT of the CAR filtered signal
fft_CAR_filtered = fft(CAR_signal, nfft) / length(CAR_signal);
power_CAR_filtered = abs(fft_CAR_filtered(1:nfft/2)).^2; % Power spectrum

% FFT of the CAR + band pass filtered signal
fft_filtered = fft(filtered_signal, nfft) / length(filtered_signal);
power_filtered = abs(fft_filtered(1:nfft/2)).^2; % Power spectrum

% Plot the power spectrum
figure;
subplot(3, 1, 1);
plot(frequencies, power_original, 'b');
title('Power Spectrum of raw EEG');
xlabel('Frequency (Hz)');
ylabel('Power');
grid on;
xlim([0 100]); % Restrict x-axis to 0-100 Hz

subplot(3, 1, 2);
plot(frequencies, power_CAR_filtered, 'r');
title('Power Spectrum After CAR Filter');
```

```
xlabel('Frequency (Hz)');
ylabel('Power');
grid on;
xlim([0 100]); % Restrict x-axis to 0-100 Hz

subplot(3, 1, 3);
plot(frequencies, power_filtered);
title('Power Spectrum After CAR + band pass Filter');
xlabel('Frequency (Hz)');
ylabel('Power');
grid on;
xlim([0 100]); % Restrict x-axis to 0-100 Hz

sgtitle('Comparison of Power Spectrum Before and After Filters');

%% AVERAGE SIGNALS (Data Exploration)
%------> average across blocks
% Preallocate the averaged dataset
averaged_data = cell(size(filtered_data));

% Loop through each subject
for i = 1:length(filtered_data)
    % Extract the data for the current subject
    subject_data = filtered_data{i}; % Dimension: 9x1215x40x6

    % First, average across the 4th dimension (blocks)
    data_avg_blocks = mean(subject_data, 4); % Dimension: 9x1215x40

    % Then, average across the 1st dimension (electrodes)
    %data_avg_electrodes = mean(data_avg_blocks, 1); % Dimension: 1x1215x40

    % Squeeze the result to remove singleton dimensions
    %averaged_data{i} = data_avg_electrodes; % Dimension: 1x1215x40
    averaged_data{i} = data_avg_blocks; % Dimension: 9x1215x40
end

% Now averaged_data contains the dataset with the last dimension averaged
averaged_signal1 = averaged_data{1}(1, :, 1,1);
disp(size(averaged_signal1));

% ------> average across all patients
% Preallocate for the final averaged data
final_averaged_data = zeros(size(averaged_data{1})); % Dimension: 9x1215x40

% Loop through each subject and sum their data
for i = 1:length(averaged_data)
    final_averaged_data = final_averaged_data + averaged_data{i};
end

% Divide by the number of subjects to compute the average
final_averaged_data = final_averaged_data / length(averaged_data);

% Display the size of the final averaged data
disp(size(final_averaged_data)); % Should be 9x1215x40


% plot final average
final_average_signal = final_averaged_data(1,:,1);

% FFT of the average signal
final_fft_average = fft(final_average_signal, nfft) / length(final_average_signal);
final_power_average = abs(final_fft_average(1:nfft/2)).^2; % Power spectrum

% Plot the smoothed signal
figure;
plot(frequencies, final_power_average, 'b', 'LineWidth', 0.5); % Original signal
title('Power Spectrum');
xlabel('Frequency (Hz)');
ylabel('Power');
grid on;
xlim([0 100]);

% Averaging across all subjects --> 10.37 Hz disappear!

%% Analisys of FFT peaks
% ------> find peaks in the FFT signal
[peak_heights, peak_indices] = findpeaks(final_power_average); % Applica findpeaks solo al segnale FFT

% Calculate corresponding frequencies
peak_frequencies = frequencies(peak_indices);

% sort peaks from the highest to the lowest
[sorted_heights, sort_order] = sort(peak_heights, 'descend');
sorted_frequencies = peak_frequencies(sort_order);

% keep just the highest peaks
num_peaks_to_keep = 10;
if length(sorted_heights) > num_peaks_to_keep
    sorted_heights = sorted_heights(1:num_peaks_to_keep);
    sorted_frequencies = sorted_frequencies(1:num_peaks_to_keep);
end

% Now sort the selected frequencies from lowest to highest
[sorted_frequencies, freq_sort_order] = sort(sorted_frequencies, 'ascend');
sorted_heights = sorted_heights(freq_sort_order);

% Save the sorted frequencies in an array
```

```
resulting_frequencies = sorted_frequencies;

% show results
disp('Frequencies corresponding to the highest peaks:');
disp(resulting_frequencies);

% plot the result with highlighted peaks
figure;
plot(frequencies, final_power_average);
hold on;
plot(sorted_frequencies, sorted_heights, 'ro', 'MarkerSize', 8);
title('FFT with the 10 highest peaks highlighted');
xlabel('Frequency (Hz)');
ylabel('Power spectrum');
legend('FFT spectrum', 'Peaks');
hold off;

%% Analisys of SNR
% Range of frequencies for SNR calculation
freq_range = 5:0.1:90; % Frequencies of interest
snr_values = zeros(size(freq_range)); % Array for SNR

% Noise window (5 Hz above and below each frequency)
noise_window = 5; % 5 Hz above and below the frequency

for i = 1:length(freq_range)
    % Find the index of the current frequency in the FFT
    [~, freq_index] = min(abs(frequencies - freq_range(i)));

    % Define the indices of the noise window
    noise_indices = (frequencies >= freq_range(i) - noise_window & ...
                     frequencies <= freq_range(i) + noise_window);
    noise_indices(freq_index) = false; % Exclude the current peak

    % Calculate the signal power (current frequency)
    signal_power = final_power_average(freq_index);

    % Calculate the noise power (average of the window)
    noise_power = mean(final_power_average(noise_indices));

    % Calculate the SNR for the current frequency
    snr_values(i) = 10 * log10(signal_power / noise_power);
end

% Plot of SNR across all frequencies
figure;
plot(freq_range, snr_values, 'LineWidth', 1.5); % Continuous line for SNR
xlabel('Frequency (Hz)');
ylabel('SNR (dB)');
title('SNR across Frequencies (5-90 Hz)');
grid on;

%% SSVEP frequency recognition and performance evaluation (simple CCA)
% Variables Initialization --> [depends on the method used (CCA or CCA-SVM)]
if svm_enabled==1
    output_data=cell(n_subjects,n_runs);
else
    outputs = zeros(1, n_runs);
    output_data=zeros(n_subjects,n_runs);
    Ave_Acc_across_groups = zeros(n_subjects, 1);
end


% Method application
for sbj = 1:n_subjects
    subject_data = filtered_data{sbj};
    EEGdata = concatenate_runs(subject_data);

%     outputs = cells(40, n_runs);
    for run = 1:n_runs
        X = EEGdata(:,:,run)';

        % SSVEP frequency recognition using CCA
        Rho = zeros(1, size(Xref, 3));

        for j = 1:size(Xref, 3) % loop along all the 40 targets
            % Extract the segment of X for the relevant time window and channels
            X_segment = X(:,:)';

            % Perform CCA between the extracted segment and the reference signal
            %[Wx, Wy, Rho(j)] = myCCA(X_segment, Xref(:,:,j)');
            [Wx, Wy, r] = myCCA(X_segment, Xref(:,:,j)');

            % Store the largest canonical correlation for this target frequency
            Rho(j) = r(1);
        end
        if svm_enabled==1
        %rho vevtor for the SVM model
        output_data{sbj,run}=Rho;
        else
        % Determine the frequency with the highest canonical correlation
        [~, outputs(run)] = max(Rho);

        end

    end
    if svm_enabled==1
```

```
        disp(['CCA-SVM: Sbj_number', num2str(sbj)]);
    else
        output_data(sbj,:)=outputs;

        % Performance evaluation
        labels = repmat(1:n_classes, 1, n_runs / n_classes);
        TotalAccuracy = compute_accuracy(labels, outputs);
        Ave_Acc_across_groups(sbj) = TotalAccuracy;

        disp(['ONLY CCA: Total accuracy(', num2str(sbj), '): ', num2str(TotalAccuracy), ' %']);

    end



end

% If CCA--> will display the results directy
% If CCA-SVM will create a cell output_data (subj x runs) where runs is #trialsx#blocks: Inside each cells there is a 40-dimension vector (this has the correlation values for each frequences of stimulation)


%% Support Vector Machine Implementation

if svm_enabled==1
        % Train & Test Set Split
        train_set_cell = output_data(:,1:200);
        test_set_cell = output_data(:,201:240);

        % Creation of Dataset from Cells (subjectxtrials, n_classes))
        train_set = cellToMatrix(train_set_cell, n_classes, n_subjects, 200); % Matrix 7000x40 contains #subjects x 200 (WE USE FIRST 5 BLOCKS FOR THE TRAIN)
        test_set = cellToMatrix(test_set_cell, n_classes, n_subjects, 40);      % Matrix 1400x40 contains #subjects x 40 (WE USE LAST 5 BLOCK FOR THE TEST)

        % Concatenate Labels to datasets  (as Row)
        labels_train = repmat(1:40, 1, 175)';
        labels_test = repmat(1:40, 1, 35)';
        train_set = [train_set, labels_train];
        test_set = [test_set, labels_test];

        %% Classification Model
        train_sett = train_set(:,1:40);
        train_labelss = train_set(:,end);
        test_sett = test_set(:,1:40);
        test_labelss = test_set(:,end);

        % Model & Cross-validation
        cross_val_model = fitcecoc(train_sett, train_labelss, 'KFold', 5); % 5-fold cross-validation

        % Train accuracy calculation
        cv_accuracy = (1 - kfoldLoss(cross_val_model)) * 100; % Percentuale di accuratezza
        disp(['Accuratezza durante la validazione incrociata: ', num2str(cv_accuracy), '%']);

        % Addestramento del modello finale sui dati di training completi
        final_model = fitcecoc(train_sett, train_labelss);

        % Prediction of the trained model
        predicted_labelss = predict(final_model, test_sett);

        % Test Accuracy
        test_accuracy = sum(predicted_labelss == test_labelss) / length(test_labelss) * 100;
        disp(['Accuratezza sul test set: ', num2str(test_accuracy), '%']);

        % Valutazione dell'overfitting
        disp('Valutazione dell''overfitting:');
        if test_accuracy < cv_accuracy
            disp('Possibile overfitting rilevato: la performance sul test set è inferiore a quella di validazione.');
        else
            disp('Nessun overfitting rilevato.');
        end

        % Matrice di confusione
        figure(8);
        confusionchart(test_labelss, predicted_labelss);
        title('Matrice di confusione');

        %% ROC CURVE
        % Calcolo delle probabilità (per curve ROC, il modello deve supportare score posteriori)
        [~, scores] = predict(final_model, test_sett);

        % Creazione della curva ROC per ogni classe
        figure(9);
        for class = 1:numel(unique(test_labelss))  % Usa test_labelss al posto di y
            [Xroc, Yroc, ~, AUC] = perfcurve(test_labelss, scores(:, class), class);
            plot(Xroc, Yroc, 'LineWidth', 2); hold on;
            disp(['AUC per la classe ', num2str(class), ': ', num2str(AUC)]);
        end
        xlabel('False Positive Rate');
        ylabel('True Positive Rate');
        title('ROC Curve per ogni classe');
        legend(arrayfun(@(x) ['Classe ', num2str(x)], unique(test_labelss), 'UniformOutput', false), 'Location', 'Best');
        grid on;


        %% PLOT EEG's signals
```

```
        figure(10);
        % Plot the signal at each preprocessing stage
        subplot(5, 1, 1);
        plot(t_raw, electrode_signal, 'b');
        title('Raw EEG Signal');
        xlabel('Time (s)');
        ylabel('Amplitude (\muV)');
        grid on;

        subplot(5, 1, 2);
        plot(t_ref, electrode_signal1, 'b');
        title('Raw EEG Signal cutted');
        xlabel('Time (s)');
        ylabel('Amplitude (\muV)');
        grid on;

        subplot(5, 1, 3);
        plot(t_ref, CAR_signal, 'g');
        title('EEG Signal After CAR Filter');
        xlabel('Time (s)');
        ylabel('Amplitude (\muV)');
        grid on;

        subplot(5, 1, 4);
        plot(t_ref, filtered_signal, 'm');
        title('EEG Signal After Band-pass + CAR Filters');
        xlabel('Time (s)');
        ylabel('Amplitude (\muV)');
        grid on;

        subplot(5, 1, 5);
        plot(t_ref, averaged_signal1, 'k');
        title('Averaged EEG Signal Across Blocks');
        xlabel('Time (s)');
        ylabel('Amplitude (\muV)');
        grid on;

        sgtitle('EEG Signal Preprocessing Stages');

        % ITR
         P = test_accuracy/100;
        ITR = (log2(n_classes)+P*log2(P)+(1-P)*log2((1-P)/(n_classes-1)))*60/duration;
        disp(['CCA-SVM Average ITR: ', num2str(ITR)]);
else
    %% Final results
    stderror = std(Ave_Acc_across_groups) / sqrt(length(Ave_Acc_across_groups));
    Ave_Acc_across_sbjs = mean(Ave_Acc_across_groups);
    plusminu = char(177);
    disp(['ONLY CCA Average accuracy: ', num2str(Ave_Acc_across_sbjs), ' ', plusminu, ' ', num2str(stderror), ' %']);

    % ITR
    P = Ave_Acc_across_sbjs/100;
    ITR = (log2(n_classes)+P*log2(P)+(1-P)*log2((1-P)/(n_classes-1)))*60/duration;
    disp(['ONLY CCA Average ITR: ', num2str(ITR)]);
end

function Xref = generate_reference_signals(fstim, Nh, t)
    % Generate reference signals for each stimulus frequency
    Xref = [];
    for i = 1:numel(fstim)
        Y = [];
        for n = 1:Nh
            tp(1,:) = sin(2*pi*(n*fstim(i))*t);
            tp(2,:) = cos(2*pi*(n*fstim(i))*t);
            Y = [Y; tp];
        end
        Xref(:,:,i) = Y';
    end
end

function X = apply_car_filter(X)
    % Apply Common Average Reference (CAR) filter
    ref = mean(X, 2);
    X = X - ref;
end

function [Wx,Wy,r] = myCCA(X,Y)
    % Ensure that X and Y have the same number of columns (time points)
    if size(X, 2) ~= size(Y, 2)
        error('The number of columns (time points) in X and Y must match.');
    end

    % Canonical Correlation Analysis
    z = [X;Y];
    C = cov(z.');
    sx = size(X,1);
    sy = size(Y,1);
    Cxx = C(1:sx, 1:sx) + 10^(-8)*eye(sx);
    Cxy = C(1:sx, sx+1:sx+sy);
    Cyx = Cxy';
    Cyy = C(sx+1:sx+sy, sx+1:sx+sy) + 10^(-8)*eye(sy);
    invCyy = inv(Cyy);
    C = inv(Cxx)*Cxy*invCyy*Cyx;

    % Eigenvalue decomposition
    [Wx,r]= eig(C);
    r = sqrt(real(r));      % Canonical correlations
```

```
    % Sort correlations
    V = fliplr(Wx);         % Reverse order of eigenvectors
    r = flipud(diag(r));    % Extract eigenvalues and reverse their order
    [r,I]= sort((real(r))); % Sort reversed eigenvalues in ascending order
    r = flipud(r);          % Restore sorted eigenvalues into descending order
    for j = 1:length(I)
        Wx(:,j) = V(:,I(j));  % Sort reversed eigenvectors in ascending order
    end
    Wx = fliplr(Wx);        % Restore sorted eigenvectors into descending order

    % Calculate Wy
    Wy = inv(Cyy)*Cyx*Wx;   % Basis in Y
    Wx = Wx(:,1:end);
end

function accuracy = compute_accuracy(labels, outputs)
    % Compute accuracy
    C = confusionmat(labels, outputs);
    accuracy = sum(diag(C)) / sum(C(:)) * 100;
end
```

Listing A.1: Code

Technical
University of
Denmark

Ørsteds Plads, Building 345C
2800 Kgs. Lyngby
Tlf. 4525 1700

www.healthtech.dtu.dk/